

1位Solution

Rank	1
Team	卷不動了
Author	TheoQiu
Topic ID	670735
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/670735>

Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このように白熱したコンペを開催してくださったorganizerに感謝します。幸運にも優勝できました。teammateにも感謝します。Kaggleコンペへ最初から最後まで参加したのは今回が初めてで、非常に貴重な経験になりました。以下が私たちのsolutionです。

CV strategy

Stateとsampling dateを組み合わせせて3 foldへ分割し、そのうちsingle fold modelを学習しました。

Input data

多くの優れたopen-source notebookと同様に、画像をleft/right viewへ分け、同じbackboneへ通して一貫性を高めました。

Training data augmentation

次のような一般的なdata augmentationを使いました。

```
def get_train_transforms(args):
    return Compose([
        HorizontalFlip(p=0.5),
        VerticalFlip(p=0.5),
        RandomRotate90(p=0.5),
        A.GaussNoise(p=0.3),
        A.RandomBrightnessContrast(
            brightness_limit=0.2,
            contrast_limit=0.2,
            p=0.75
        ),
        A.HueSaturationValue(
            hue_shift_limit=10,
            sat_shift_limit=20,
            val_shift_limit=20,
            p=0.5
        ),
        A.CLAHE(clip_limit=2.0, tile_grid_size=(8, 8), p=0.3),
        A.ColorJitter(
            brightness=0.2,
            contrast=0.2,
            saturation=0.2,
            hue=0.1,
            p=0.75
        ),
        Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        ),
        Resize(args.img_size, args.img_size),
        ToTensorV2()
    ])
```

left/right subimageへ分けた後、各subimageへ独立にaugmentationを適用しました。さらに、小さなrandom scaling後にblack pixelで補完し、cameraごとのfocal lengthとviewing angleの差を再現しました。

```
if random.random() < 0.2:
    background_image = np.full_like(image, 0)
    resize_retio = random.uniform(0.85, 1.0)
    image = cv2.resize(image, None, fx=resize_retio, fy=resize_retio, interpolation=cv2.INTER_CUBIC)
    h, w, _ = image.shape
    bg_h, bg_w, _ = background_image.shape
    top = random.randint(0, bg_h - h)
    left = random.randint(0, bg_w - w)
    background_image[top:top + h, left:left + w] = image
    image = background_image
```

Backbone selection

backboneにはDINOv3を使いました。このコンペの多くの上位teamも同じapproachだったと思います。続いて、left/right subimageから連結したfeatureをsingle multi-head self-attention layerで相互作用させ、融合featureをMLPから出力しました。

Head design

各speciesのbiomassを、3 linear layerからなる**5つのregression head**で独立に予測しました。training中に物理constraintは課しませんでした。これによりmodelのtraining setへの依存を減らせたと考えています。

多くのsolutionと異なり、さらに5つの**classification head**を設計しました。このtaskはdensity mapのないcrowd counting (CC) problemに似ているため、CC分野の多くの研究を読みました。UEPNet [1]のinterval partition schemeを参考に、training setを使ってspeciesごとに7 intervalへ分けました。

```
BORDERS_DICT = {
    'Dry_Clover_g': [1.6e-05, 3.9, 10.5353, 20.6523, 37.5911, 71.7865],
    'Dry_Dead_g': [1.6e-05, 6.1407, 13.1192, 23.277, 38.8581, 83.8407],
    'Dry_Green_g': [1.6e-05, 13.4232, 27.0782, 45.5236, 79.834, 157.9836],
    'Dry_Total_g': [1.6e-05, 23.4907, 41.1, 61.1, 96.8288, 185.7],
    'GDM_g': [1.6e-05, 16.5143, 30.507, 49.5585, 81.0, 157.9836],
}
```

分割したintervalに基づき、biomassが属するintervalを予測するclassification headをspeciesごとに追加しました。これによりLBとPBの両方で約0.03改善し、CVとも非常によく整合しました。理想的にはintervalを十分細かく分ければ、interval inference値からbiomassを正確に予測できます（density map/segmentation mapのないこのコンペでは明らかに難しすぎます）。

Loss function

最終提出modelは、regressionへ一般的なSmoothL1 Loss、classificationへCross-Entropy lossを使いました。各biomassのlossは、 R^2 計算でコンペが定めたweightに従ってscaleしました。

```
class WeightedBiomassLoss(nn.Module):
    def __init__(self, loss_weights_dict):
```

```

super(WeightedBiomassLoss, self).__init__()

self.criterion = nn.SmoothL1Loss()
self.cls_loss = nn.CrossEntropyLoss(reduction='none')
self.cls_weight = 0.3
self.weights = loss_weights_dict

def forward(self, predictions, targets, predictions_cls=None, cls_labels=None):
    total_cls_loss = None
    pred_green, pred_dead, pred_clover, pred_gdm, pred_total = predictions
    if predictions_cls is not None:
        assert cls_labels is not None
        pred_green_cls, pred_dead_cls, pred_clover_cls, pred_gdm_cls, pred_total_cls =
predictions_cls
        cls_green_loss = self.cls_loss(pred_green_cls, cls_labels[:, 0])
        cls_dead_loss = self.cls_loss(pred_dead_cls, cls_labels[:, 1])
        cls_clover_loss = self.cls_loss(pred_clover_cls, cls_labels[:, 2])
        cls_gdm_loss = self.cls_loss(pred_gdm_cls, cls_labels[:, 3])
        cls_total_loss = self.cls_loss(pred_total_cls, cls_labels[:, 4])
        total_cls_loss = (
            self.weights['green_loss'] * cls_green_loss.mean() +
            self.weights['dead_loss'] * cls_dead_loss.mean() +
            self.weights['clover_loss'] * cls_clover_loss.mean() +
            self.weights['gdm_loss'] * cls_gdm_loss.mean() +
            self.weights['total_loss'] * cls_total_loss.mean()
        )

    true_green = targets[:, 0].unsqueeze(-1) # Shape [batch, 1]
    true_dead = targets[:, 1].unsqueeze(-1) # Shape [batch, 1]
    true_clover = targets[:, 2].unsqueeze(-1) # Shape [batch, 1]
    true_gdm = targets[:, 3].unsqueeze(-1) # Shape [batch, 1]
    true_total = targets[:, 4].unsqueeze(-1) # Shape [batch, 1]

    loss_green = self.criterion(pred_green, true_green)
    loss_dead = self.criterion(pred_dead, true_dead)
    loss_clover = self.criterion(pred_clover, true_clover)
    loss_gdm = self.criterion(pred_gdm, true_gdm)
    loss_total = self.criterion(pred_total, true_total)

    total_loss = (
        self.weights['total_loss'] * loss_total +
        self.weights['gdm_loss'] * loss_gdm +
        self.weights['green_loss'] * loss_green +
        self.weights['dead_loss'] * loss_dead +
        self.weights['clover_loss'] * loss_clover # +
    )
    if predictions_cls is not None:
        total_loss = total_loss + self.cls_weight * total_cls_loss
    return total_loss, total_cls_loss

```

最終modelには入れなかった追加実験もありますが、PBにより、この過程での設計が有意義だったことが分かりました。例えばregression optimizationにepsilon-insensitive L1 lossを使う方法です。biomassの不均衡分布を踏まえ、**「大きな値は大きな誤差、小さな値は小さな誤差」**となることを期待しました。そのためlabelに基づく次のlossを設計し、PBでbest single-model resultを得ました。

```

class EpsilonInsensitiveLoss(nn.Module):
    def __init__(self, eps_point=20, max_eps=5, reduction='mean'):
        super().__init__()
        self.eps_point = eps_point
        self.reduction = reduction

```

```

self.scale_ratio = 0.1
self.max_eps = max_eps

def make_epsilon(self, target):
    epsilon = torch.zeros_like(target)
    epsilon = torch.where(
        target <= self.eps_point, torch.ones_like(target), epsilon
    )
    epsilon = torch.where(
        target > self.eps_point, target * self.scale_ratio, epsilon
    )
    epsilon = torch.clamp(epsilon, max=self.max_eps)

    return epsilon

def forward(self, pred, target):
    epsilon = self.make_epsilon(target.detach())
    abs_diff = torch.abs(pred - target)
    loss = torch.relu(abs_diff - epsilon)

    if self.reduction == 'mean':
        return loss.mean()
    elif self.reduction == 'sum':
        return loss.sum()
    return loss

```

Training strategy

コンペ序盤にさまざまなstrategyを試し、最も効果的だった2-stage trainingを最終的に選びました。

- **Stage 1:** DINOをfreezeし、fusion layerとMLP headを学習。
- **Stage 2:** model全体をfine-tune。

私たちの設定ではLBでもCVでもDINO-LargeとDINO-Hugeのperformanceはほぼ同じでした。計算資源が限られる場合、model sizeを増やすよりinput sizeを増やす方が有効でした。そのため後半のbest single modelは主に1024px+Largeで学習しました。

DINOv3-Large single-model result

exp_name	input_size (single view)	LB	PB
Baseline-large	512	0.70	0.60
+cls	512	0.73	0.63
+cls	1024	0.74	0.64
+aug_scale + cls	1024	0.74	0.65
+cls + ϵ-insensitive loss	512	0.71	0.66

Test time

test時のonline trainingで0.02を超えるgainが得られました。

1. LBで良く、かつ多様性のある4 single modelを選び、test setのpseudo-labelを生成しました。
2. pseudo-labelを使い、DINO-Large 2 modelを反復学習しました。通常の学習と異なり、training setとtest set全体を使い、CVは行わず、最後の複数epochのSWA ensemble modelを採用しました。pseudo-labelへoverfitしないよう、pseudo-label学習後にtraining setだけで数epoch学習しました。
3. 前段のDINO-Large 2 modelで2回目のpseudo-labelを生成し、同じstrategyでDINO-Base 2 modelを学習しました。
4. DINO-Large 2 modelとDINO-Base 2 modelの推論結果を、それぞれ0.4と0.6のweightで融合しました。submission notebookは[Final_Inference_Notebook](#)で公開しています。

Post-processing

LBとPBを僅かに改善する単純なpost-processingを使いました。

```
out_green = self.head_green(combined)
out_dead = self.head_dead(combined)
out_clover = self.head_clover(combined)
out_gdm = self.head_gdm(combined)
out_total = self.head_total(combined)

out_clover = out_clover * 0.8
if out_dead > 20:
    out_dead *= 1.1
elif out_dead < 10:
    out_dead *= 0.9

out_gdm = 0.5*out_gdm+ 0.5*(out_green+out_clover)

pred_total1 = out_green + out_clover + out_dead
pred_total2 = out_gdm + out_dead
out_total = 0.5*out_total + 0.5*pred_total1 + 0.0*pred_total2
```

最終score

exp_name	LB	PB
PBでbest	0.77	0.68
submitted	0.77	0.67

うまくいかなかったideaと考察

1. さまざまなsemi-supervised scheme。実装とtuningに最後の数日をほぼ使いましたが、直接的なhard semi-supervised approachを超えませんでした。

2. speciesごとに1 modelを学習し、5 modelをmergeする方法。
3. trainingへ物理constraintを導入する方法、または複数headを学習して残りのbiomassを計算する方法。
4. dropoutの除去。regression taskではtraining/inference間のvarianceが一致しなくなるためdropoutは通常不利ですが、実験ではdropoutが非常によく、除去を大幅に上回りました。
5. 未label dataとexternal datasetの追加生成。
6. training imageへのshadow追加。
7. ranking loss。小さなtraining dataでbiomassをregion間比較させるため、training image x をrandom cropして x_{crop} を作り、 y_{crop} という内部順位を使いました。さらにlabelから画像間順位も計算しました。CVでは非常によく、以前のbest modelを一時超えましたが、LBは悪く、再びoverfittingした可能性があります。
8. L1 lossを他のさまざまなlossへ置換。
9. 複数解像度patchの入力 ($[1 \times 2, 2 \times 4, 4 \times 8, \dots]$)。最終的には良いCV scoreを得ましたが、激しく振動しました。LB/PBも良くなり、特にPBIは0.6未満でした。
10. そのほか多数。

最後に、teammateの[@pingfan](#)を祝福します。Kaggle communityに新しいGMが誕生しようとしています！

[1] Wang, Changan, Qingyu Song, Boshen Zhang, Yabiao Wang, Ying Tai, Xuyi Hu, Chengjie Wang, Jilin Li, Jiayi Ma, and Yang Wu. “Uniformity in heterogeneity: Diving deep into count interval partition for crowd counting.” ICCV, pp. 3234–3242, 2021.

補足Q&A

Post-processing値をどう決めたか

V1kstrand (質問) : post-processingの設定は複数の値をsweepしたのでしょうか。それともdata上の仮定に基づきますか。

TheoQiu (著者) : 一部はopen-source notebookを参考にし、一部はCVとLB scoreから導きました。

他backboneとpseudo-labelingの役割

benkerrouche abdelbasset (質問) : 他のbackboneも試しましたか。また最終approachのpseudo-labelingは安定化とself-distillationのためでしょうか。

TheoQiu (著者) :

1. 私はSwin seriesも試し、teammateはDINOv2を試しましたが、どちらもDINOv3ほど効果的ではありませんでした。
2. はい。これは実際にself-distillation strategyです。